University of Exeter

ECM2433: The C Family
More types

Richard Everson

Integer division

Division of integers rounds towards zero

```
int i = 56;
int j = -56;
int denominator = 10;

printf("%d  %d\n", i/denominator, j/denominator);
```

```
5  -5
```

Promotion and casting

Types are (implicitly) promoted to high precision before combination with arithmetic operators

```
int i = 56;
int j = -56;
float denominator = 10.0;

printf("%f %f\n", i/denominator, j/denominator);
```

```
5.600000   -5.600000
```

Promotion and casting

Types are (implicitly) promoted to high precision before combination with arithmetic operators

```
int i = 56;
int j = -56;
float denominator = 10.0;

printf("%f %f\n", i/denominator, j/denominator);
```

```
5.600000   -5.600000
```

Promotions:

```
char => int
int =>  float
float => double
```

Promotion and casting

Types can be explicitly **cast** to another type

```
int i = 56;
int j = -56;
int denominator = 10;
printf("%f %f\n", i/(float)denominator, (float)j/denominator);
```

```
5.600000   -5.600000
```

```
float denominator = 10.0;
printf("%d %d\n", i/(int)denominator, j/(int)denominator);
```

```
5   -5
```

Type conversions in C

```
#include <stdio.h>
int main ()
{
    char c = 'a';
    int n = 10*c;       /* c promoted to int before multiplication */
    float f = c/12.0;   /* c promoted to float before division */

    printf(”c = %c and %d    n = %d    f = %f\n", c, c, n, f);

    return 0;
}
```

```
c = a and 97    n = 970    f = 8.083333
```

C is weakly typed: might be bad

University of Exeter

"weakly typed" = supports implicit type conversions

```
unsigned short int x = 65535;    // 2^16-1

short int          y = x;
char               c = x;
float              f = x;

printf("x = %d\n", x);
printf("y = %d\n", y);
printf("c = %c\n", c);
printf("f = %f\n", f);
```

```
x = 65535
y = -1
c = ▒
f = 65535.000000
```

C is weakly typed: probably bad

University of Exeter

"weakly typed" = supports implicit type conversions

```
float f = 123456.789123;

unsigned short int x = f;
short int          y = f;
char               c = f;

printf("x = %d\n", x);
printf("y = %d\n", y);
printf("c = %c\n", c);
printf("f = %f\n", f);
```

```
x = 57920
y = -7616
c = @
f = 123456.789062
```

Notice: no compile-time or runtime errors are generated!

New types for old: typedef

Define a synonym for a type: `typedef base new`

Example:

```
typedef float Temperature_t;
Temperature_t today = 8;
Temperature_t tomorrow = today - 2;
```

C defines `size_t` for the sizes of variables:

```
typedef unsigned long int size_t;

size_t n = sizeof(double);    //  8 bytes
```

struct

- Compound variables can be defined with a struct.
- Members are accessed with dot notation.

```
struct WEATHER {
       float temperature;  // Degrees C
       bool raining;       // Is it?
       int visibility;     // Poor (0) to good (10)
};

struct WEATHER today;
today.temperature = 8; // Implicit promotion int -> float
today.visibility = 3;
today.raining = true;
```

struct and typedef

```
typedef float Temperature_t;
typedef struct WEATHER {
        Temperature_t;      // Degrees C
        bool raining;       // Is it?
        int visibility;     // Poor (0) to good (10)
} weather_t;

weather_t today;

today.temperature = 8; // Implicit promotion int -> float
today.visibility = 3;
today.raining = true;
```

Summary

- Integer division rounds towards zero
- Types are **implicitly promoted** when combined with arithmetic operators
- Types can be **explicitly cast** to another type
- **Beware** the consequences of **weak typing**
- **structs** allow **compound** types
- **typedef** for **readability**

- Read about **enums** and **unions**